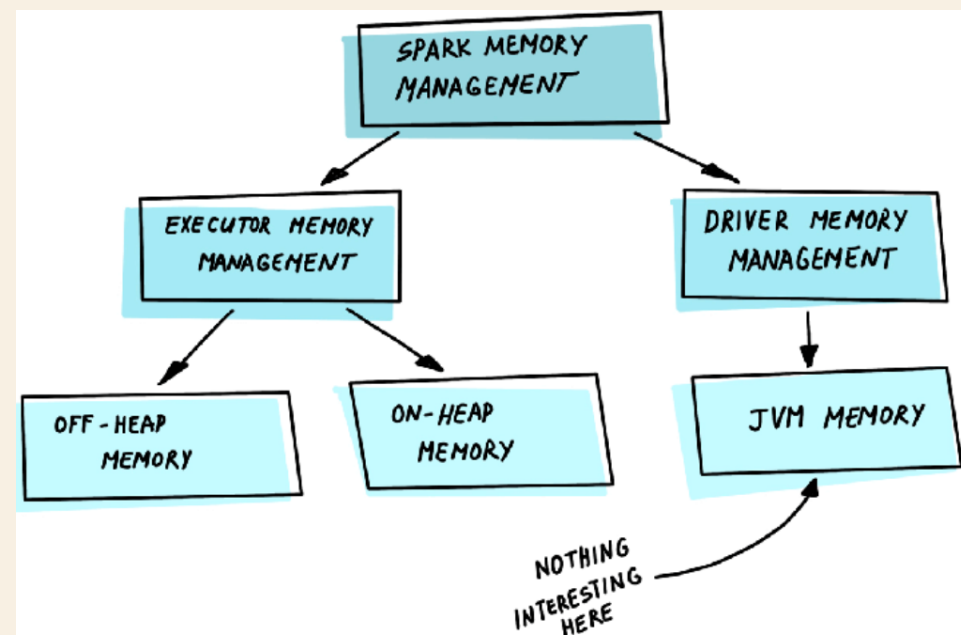


SPARK MEMORY MANAGEMENT

- ❑ Spark's performance heavily depends on how well it manages memory. Unlike traditional systems, Spark divides memory into regions for **execution** and **storage**. Understanding this is crucial for avoiding out-of-memory (OOM) errors and optimizing your job performance.
- ❑ spark is a execution framework which provides a means to process data in parallel. Unlike Hadoop, Spark applications are **memory** heavy. This leads to the need for understanding how memory management is done in spark, this will help you in tuning the configurations of spark to make the best out of the resources available. **Executor** is where all the computing is done in spark, hence, we will be focusing on **executor memory** management.

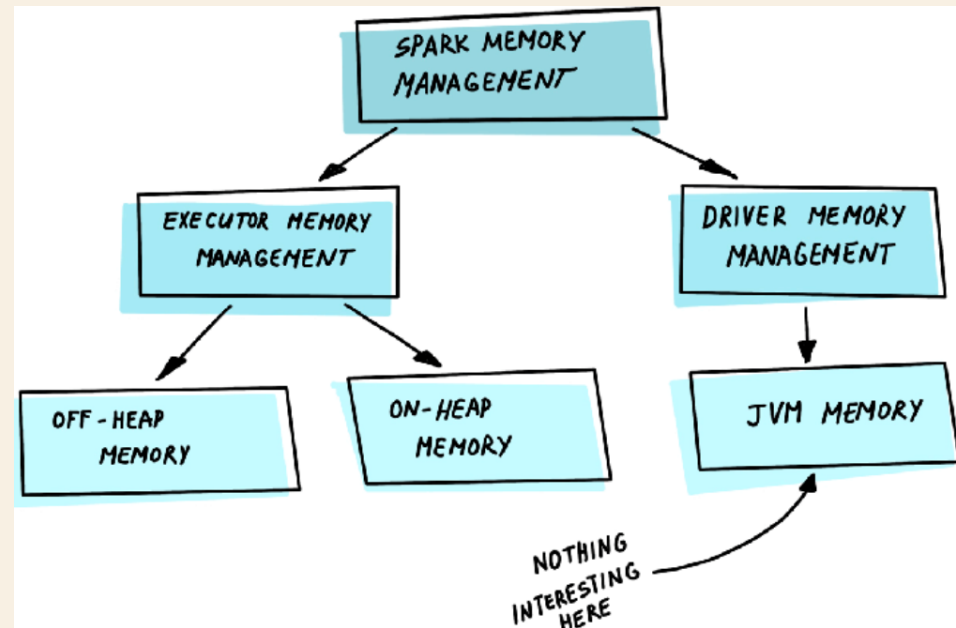


When the Spark application is launched, the Spark cluster will start two processes – **Driver and Executor**.

The driver is a master process responsible for creating the Spark context, submission of Spark jobs, and translation of the whole Spark pipeline into computational units – tasks. It also coordinates task scheduling and orchestration on each Executor.

Driver memory management is not much different from the **typical JVM process** and therefore will not be discussed further.

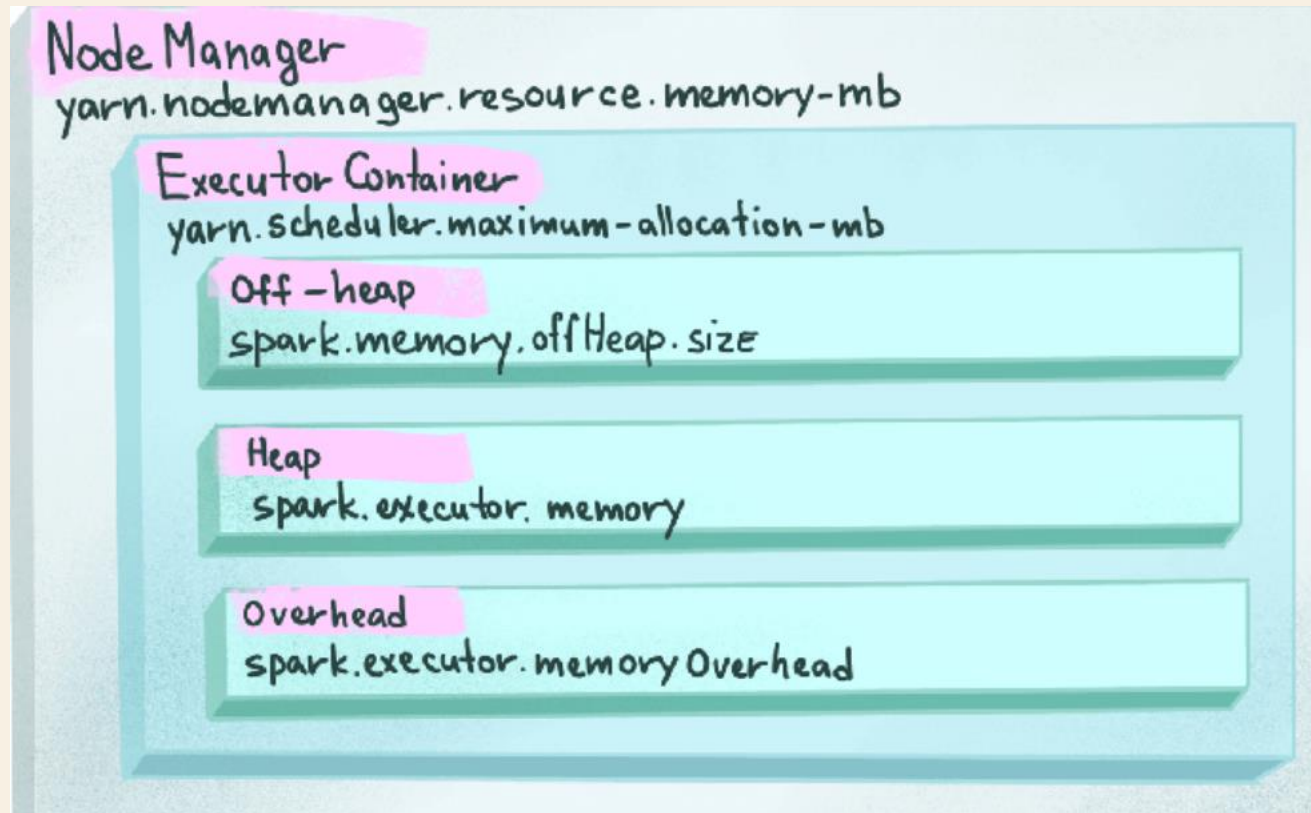
The **executor** is responsible for performing specific computational tasks on the worker nodes and returning the results to the driver, as well as providing storage for RDDs. And its internal memory management is very interesting.



When **submitting a Spark job in a cluster with Yarn**, Yarn allocates Executor containers to perform the job on different nodes.

ResourceManager handles memory requests and allocates executor containers up to the maximum allocation size settled by **yarn.scheduler.maximum-allocation-mb** configuration. Memory requests higher than the specified value will not take effect.

On a single node, it is done by **NodeManager**. NodeManager has an upper limit of resources available to it because it is limited by the resources of one node of the cluster. In Yarn, it is set up by **yarn.nodemanager.resource.memory-mb** configuration. It is the amount of physical memory per NodeManager, in MB, which can be allocated for yarn containers.



One **ExecutorContainer** is just one JVM. The entire ExecutorContainer memory area is divided into three sections:

➤ Heap memory

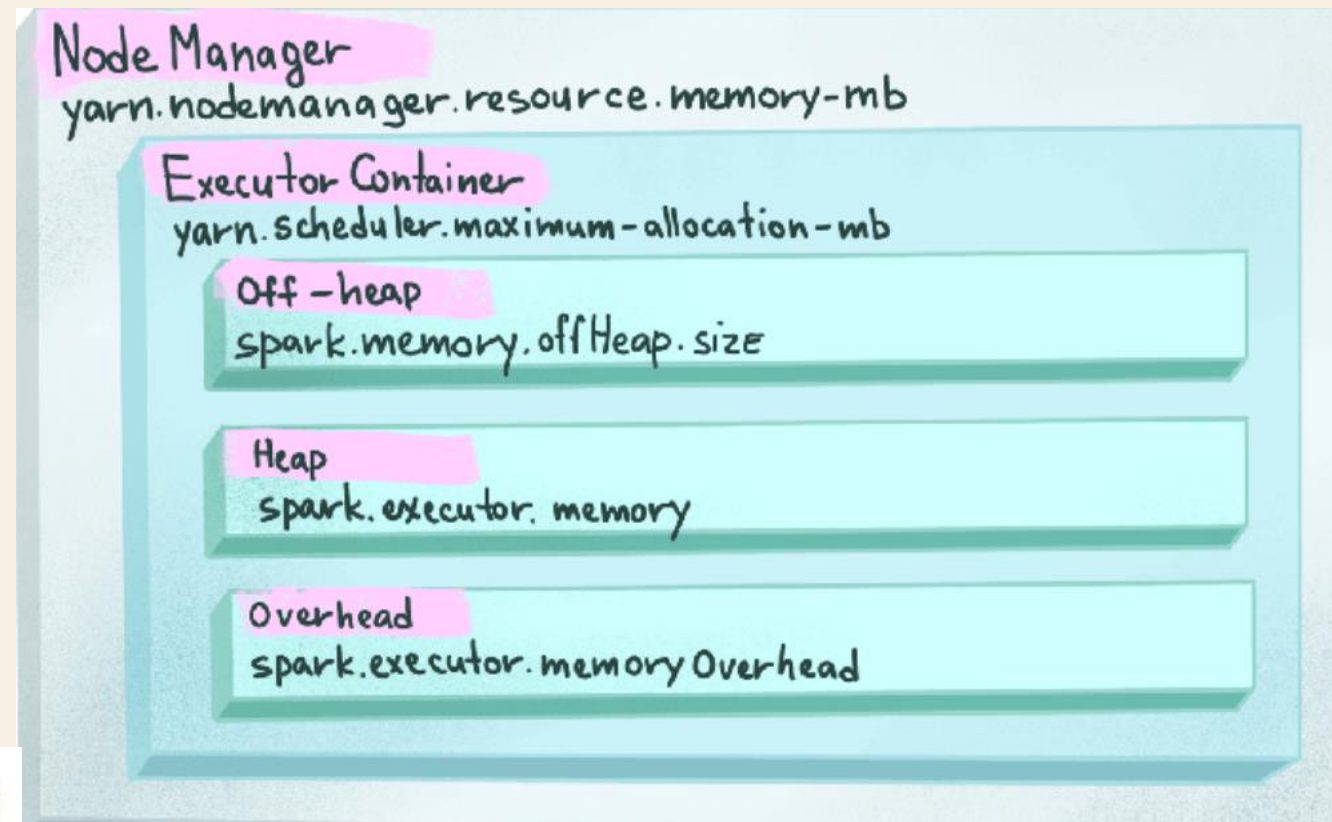
This is the memory that should be known to most Spark developers. This is the memory size specified by **--executor-memory** during submitting the Spark application or by setting **spark.executor.memory**. It's the maximum **JVM heap memory(Xmx)**. Objects here are bound by the garbage collector(GC). Further, I will call this part of memory **Executor Memory**.

➤ Off-heap memory

Spark makes it possible to use off-heap storage for certain operations. And you can provide the size of the off-heap memory that will be used by your application.

➤ Overhead memory

This is a seatbelt for the Spark execution pipelines. It is used for the various internal Spark overheads.



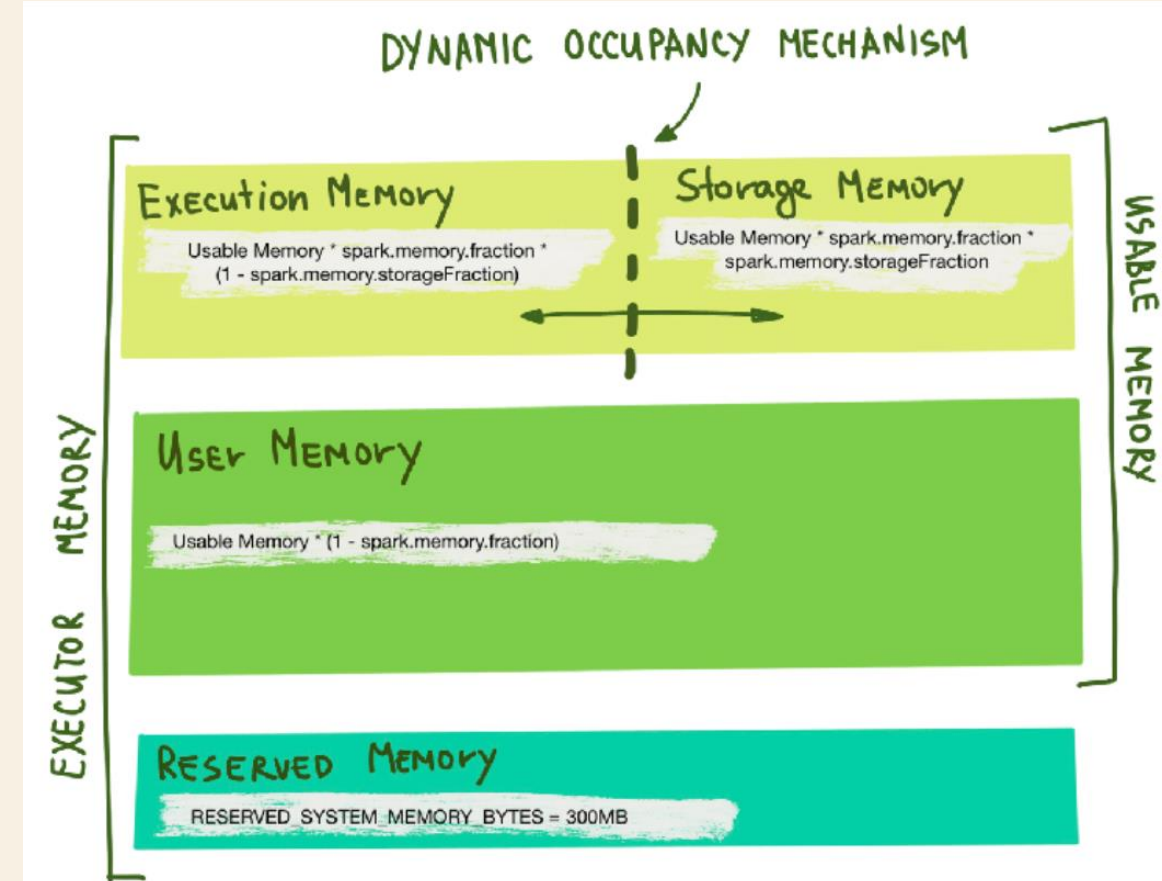
Spark provides an interface for memory management via **MemoryManager**. It implements the policies for dividing the available memory across tasks and for allocating memory between storage and execution.

MemoryManager has two implementations —

StaticMemoryManager and **UnifiedMemoryManager**. **StaticMemoryManager** was used before 1.6 and still supported and can be configured with `spark.memory.useLegacyMode` parameter. It is a legacy and will not be described further.

UnifiedMemoryManager is the default **MemoryManager** in Spark since 1.6. And we going to dive into its high-level implementation.

As we have already mentioned, the amount of memory available to each executor is controlled by `spark.executor.memory` configuration. To use and manage this part of memory more efficiently, Spark has logically and physically divided this part of the memory.



Blocks

The bulk of the data living in Spark applications is physically grouped into blocks. They are transferable objects, used as inputs to Spark tasks, as returned as outputs, they also used as intermediate steps in the shuffle process, and to store temporary files.

The **BlockManager** is the **key-value store** for blocks in Spark. BlockManager works as a local cache that runs on every node of the Spark application, i.e. driver and executors. **They can be stored on disk or in memory (on/off-heap)**, either locally or remotely, for some time. After that, they are evicted.

What is the basic idea behind blocks? Among other things, blocks help achieve more concurrency. For example, if a given block can be extracted from 4 different executor nodes, it would be faster to extract it from them in parallel.

Big Picture of Spark Memory Management

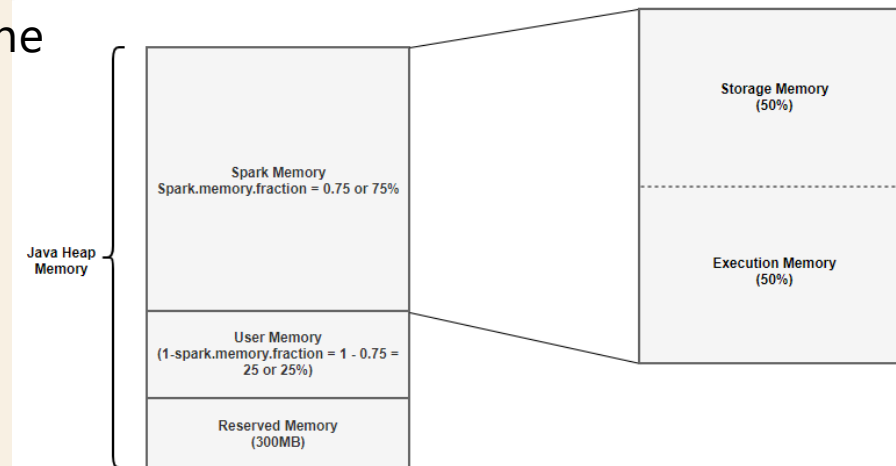
Spark's memory is divided into two main regions inside the JVM heap for execution:

1. **Execution Memory** → for shuffles, joins, aggregations, and other processing tasks.

2. **Storage Memory** → for caching & broadcasting datasets.

Why Java Heap memory?

- Spark runs on the JVM. All normal Java/Scala objects (Rows, InternalRows, collections, serializers' buffers) are allocated on the heap by default. So the heap is the natural place for Spark data structures and task objects.
- Garbage collection (GC): heavy allocation of many small objects (or huge objects) causes frequent GC pauses if heap is too full or objects are short-lived. That affects task latency.



- Tungsten / off-heap: Spark introduced the Tungsten engine to reduce Java object overhead by using compact binary representations and – optionally – off-heap memory. Off-heap (direct) memory avoids JVM GC for those **buffers, but requires explicit configuration** (`spark.memory.offHeap.enabled` and `spark.memory.offHeap.size`) and careful handling.

Note: Netty / network buffers and some native libraries may use direct (off-heap) memory, which is separate from the JVM heap. That memory still contributes to the process RSS, so you must leave headroom.

The JVM heap is split into these regions:

1. Reserved memory (~300 MB)

This is the memory reserved by the system, and its size is hardcoded. A small fixed area Spark sets aside for internal housekeeping (native libs, metadata, safely handling tiny internal allocations).

This is not part of Spark's dynamic memory pools and acts like a safety buffer so Spark internals don't immediately OOM.

Calculation for 4GB : **Reserved Memory = 300MB**

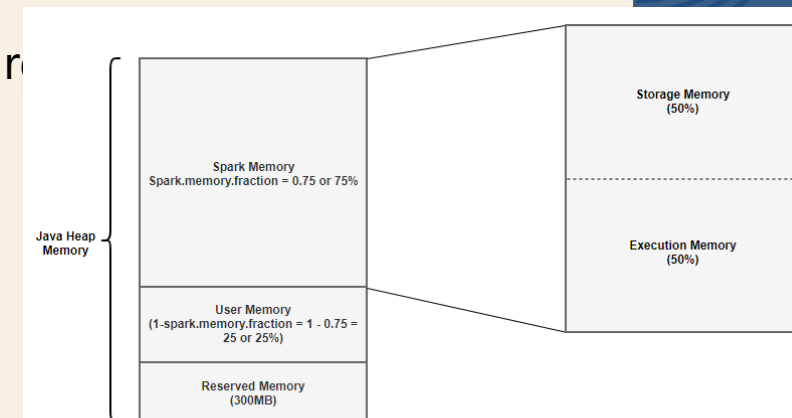
2. User memory

Holds your application objects that Spark doesn't manage: user variables, collections created in tasks, thread stacks, third-party libraries, accumulator objects, etc.

This is not managed by Spark's Unified Memory Manager. If your code allocates large objects, it can run out of heap.

User Memory = (Java Heap — Reserved Memory) * (1.0 — spark.memory.fraction)

Calculation for 4GB : **User Memory = (4024MB — 300MB) * (1.0—0.75) = 949MB**



The JVM heap is split into these regions:

3.Unified memory (managed by Spark)

This is the dynamic region controlled by Spark settings and used for Execution and Storage.

*Formula : (Java Heap — Reserved Memory) * spark.memory.fraction*

*Calculation for 4GB : (4096MB -300MB) * 0.75 = 2847MB*

- **Execution memory:** used for task processing (shuffle buffers, sort/aggregate/spill, map-side buffers). This memory region is used by Spark for objects created during execution of a task. For example, it is used to store hash table for hash aggregation step, it is used to store shuffle intermediate buffer on the Map side in memory etc,. This pool also supports spilling on disk if not enough memory is available, but the blocks from this pool cannot be forcefully evicted by other tasks.

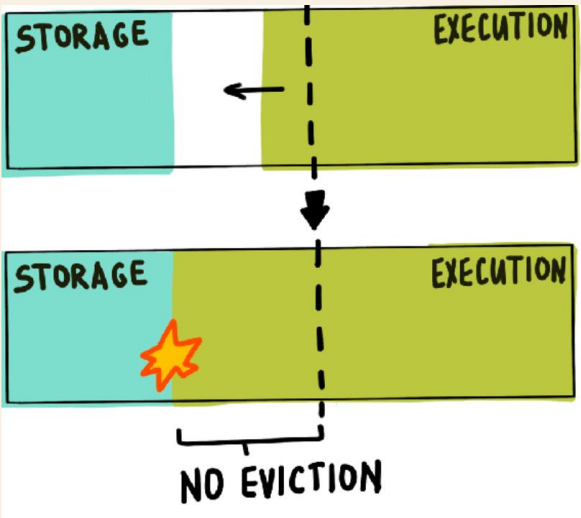
Execution Memory = (Java Heap – Reserved Memory) * spark.memory.fraction * (1.0 – spark.memory.storageFraction)

- **Storage memory:** Storage memory is used for storing all of the cached data, broadcast variables are also stored here. Any persist option which includes MEMORY in it, spark will store that data in this segment. Spark clears space for new cache requests by removing old cached objects based on Least Recently Used (LRU) mechanism. Once the cached data is out of storage, it is either written to disk or recomputed based on configuration. Broadcast variables are stored in this segment with MEMORY_AND_DISK persistent level.

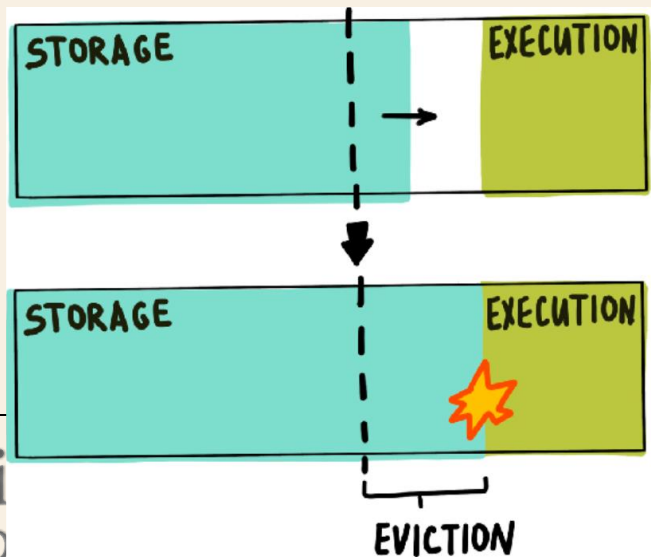
Storage Memory = (Java Heap – Reserved Memory) * spark.memory.fraction * spark.memory.storageFraction

Dynamic occupancy mechanism.

Execution and Storage have a shared memory. They can borrow it from each other. This process is called the **Dynamic occupancy mechanism**.



There are two parts of the shared memory — the Storage side and the Execution side. The shared Storage memory can be used up to a certain threshold. In the code, this threshold is called **onHeapStorageRegionSize**. This part of memory is used by Storage memory, but only if it is not occupied by Execution memory. Storage memory has to wait for the used memory to be released by the executor processes. The default size of **onHeapStorageRegionSize** is all Storage Memory



When Execution memory is not used, Storage can borrow as much Execution memory as is available until execution reclaims its space. When this happens, cached blocks will be evicted from memory until sufficient borrowed memory is released to satisfy the Execution memory request.

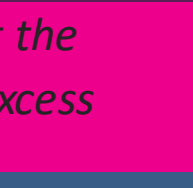
The creators of this mechanism decided that **Execution memory has priority over Storage memory**. They had reasons to do so — the execution of the task is more important than the cached data, and the whole job can crash if there is an OOM in the execution.

Dynamic occupancy mechanism.

© Arvind Kumar
#60DaysofSpark

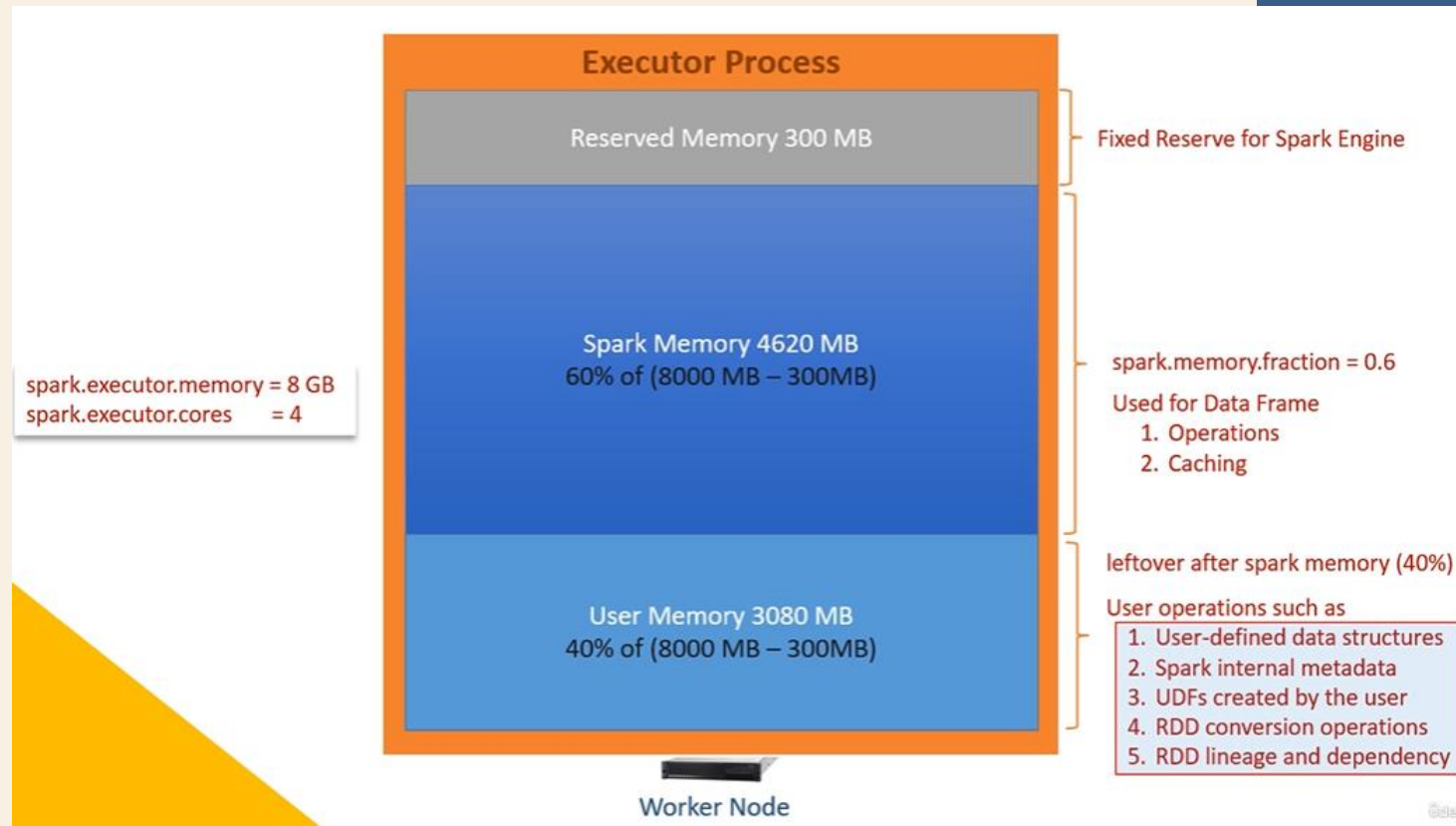
If space on both sides is insufficient and within the appropriate boundaries, it is evicted according to their respective storage levels using the LRU algorithm.

The eviction process has its overhead. The cost of memory eviction depends on the storage level. `MEMORY_ONLY` may be the most expensive because it has to be recalculated. `MEMORY_AND_DISK_SER` is the opposite. On this storage level, the format of data stored at runtime is compact and the overhead for serialization is low and it only includes disk I/O.



Few things to keep in mind about storage and execution memory:

1. Storage memory can borrow space from execution memory only if blocks are not used in Execution memory.
2. Execution memory can also borrow space from Storage memory if blocks are not used in Storage memory.
3. If blocks from Execution memory is used by Storage memory and Execution needs more memory, it can forcefully evict the excess blocks occupied by Storage Memory
4. If blocks from Storage Memory is used by Execution memory and Storage needs more memory, it cannot forcefully evict the excess blocks occupied by Execution Memory, it will end up having less memory area. It will wait until Spark releases the excess blocks stored by Execution memory and then occupies them.



Common causes of OOMs and how to fix them

Large `driver.collect()` or large local objects → move processing to distributed ops or increase driver/executor memory.

Very large shuffle data (few partitions) → increase partitions, use repartition/coalesce, tune shuffle configs.

Skewed keys → cause one executor to process huge amount of data → fix with salting or adaptive query planning.

Caching too many partitions without eviction → use `MEMORY_AND_DISK` or increase storage pool.

Too many small objects → use Kryo serialization, use DataFrame (columnar) or Tungsten binary formats to reduce object churn.

Insufficient memory overhead (on YARN) → set `spark.executor.memoryOverhead` (YARN default is $\max(384\text{MB}, 0.10 * \text{executorMemory})$) so native/off-heap usage has headroom.

Practical tuning tips

- Start with `spark.executor.memory` large enough for workload, then compute unified & user memory (like example).
- Prefer Spark SQL/DataFrame APIs (internal binary InternalRow) to avoid huge numbers of Java objects.
- Use Kryo serializer (`spark.serializer = org.apache.spark.serializer.KryoSerializer`).
- Persist with `MEMORY_AND_DISK_SER` or `MEMORY_ONLY_SER` when memory pressure is high (serialized cache uses much less memory).
- Monitor the Spark UI: storage tab shows cached block sizes; Executors tab shows GC time – high GC means too little heap or too many objects.
- For heavy aggregations / shuffles, increase shuffle partitions and/or increase `spark.memory.fraction` (careful – that reduces user memory).
- Consider enabling off-heap memory for specialized workloads (allocate `spark.memory.offHeap.size`) – only if you understand its tradeoffs.

Follow <https://www.linkedin.com/in/arvind-kumar-206020182/> for #60DaysOfSpark and Data Engineering related content

© Arvind K
#60DaysOfSp



The banner features logos for Databricks, PySpark, SQL, AWS, and Spark. The text reads: **ARVIND KUMAR**
DATA ENGINEER WITH EXPERTISE IN PYSPARK, HON, ETL, DATA ARCHITECTURE, AND CLOUD
SEND ME A DM
ARVINDKUMAR.WORK07@GMAIL.COM



Arvind Kumar ✓ He/Him
Data Engineer | Databricks | Python | SQL | NoSQL | MongoDB | Splunk | Cloud | ETL Pipelines | Apache Spark | Big Data
Hyderabad, Telangana, India · [Contact info](#)
3,136 followers · 500+ connections

[Open to](#) [Add profile section](#) [Enhance profile](#) [Resources](#)

 ADP
 National Institute of Technology, Tiruchirappalli

© Arvind Kumar
#60DaysOfSpark

